SmartResolve: AI-Based Smart Helpdesk Ticketing System

Project Summary & Agile Engineering Report | Coursera Guided Project Submission

Title of Project: SmartResolve (AI-Based Helpdesk Ticketing)	**Software / Tool Used: Asana** (Agile Kanban & Sprint Management)
Authors: Mayank Sawant & Sayali Rajput	**Institution:** AI & Data Science, MPSTME, SVKM's NMIMS
Project Domain: Enterprise IT Service Management (ITSM)	**Code Repository:** github.com/themayankguy/smart-resolve

1. Software / Tool Used: Asana (Project & Sprint Tracking)

Asana was utilized as the primary software project management and collaboration tool throughout the entire lifecycle of the **SmartResolve** project. Asana enabled the development team to apply Agile/Scrum principles to artificial intelligence engineering, breaking down the complex natural language processing pipeline, deep learning anomaly detector, backend RESTful microservices, and interactive frontend into discrete user stories and deliverables.

The project workspace in Asana was organized around a structured Kanban board with five core stages: **Backlog** (conceptual items and incoming requirements), **Ready** (refined tasks scheduled for implementation), **In Progress** (active feature engineering and model training), **Review & Validation** (code review, benchmark verification, and test execution), and **Done** (formally verified and merged features).

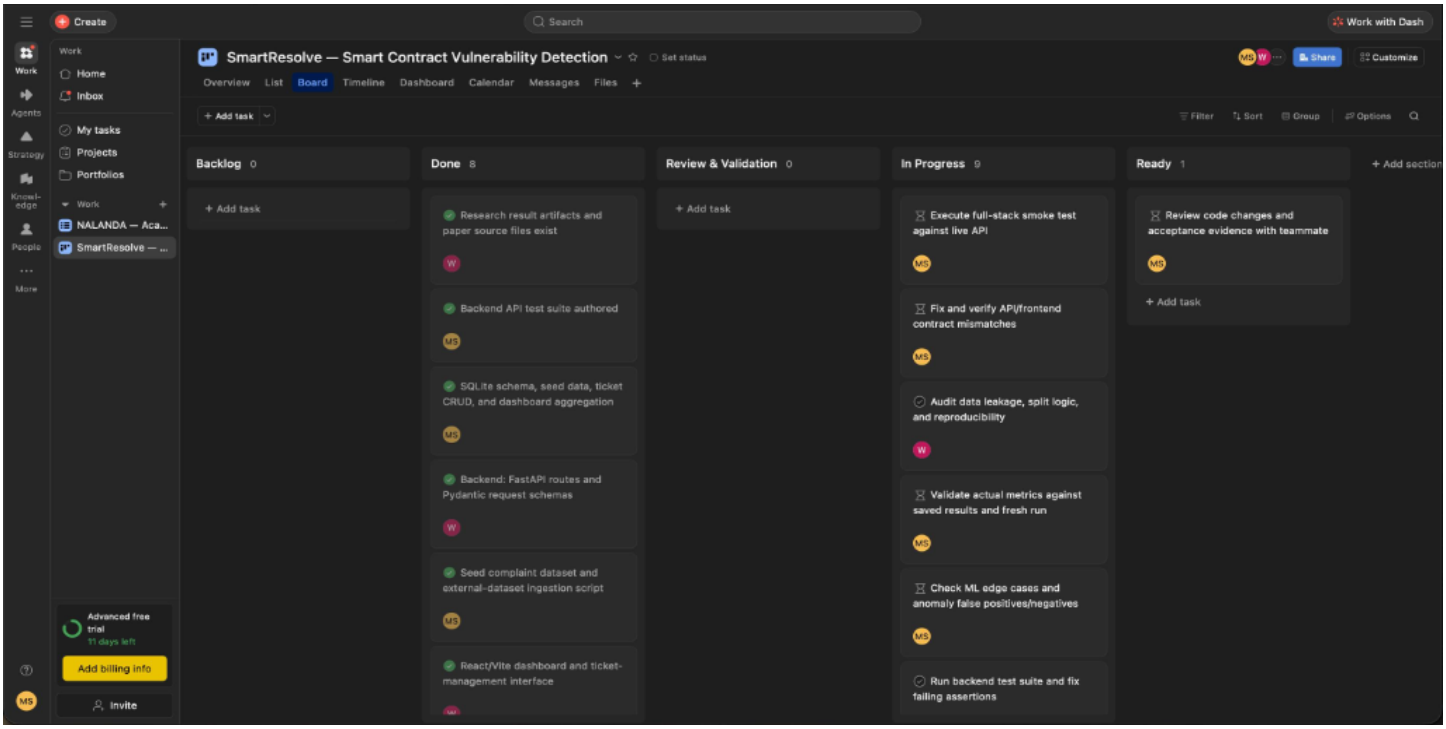


Figure 1: Screenshot of Tool Mentioned (Asana): *Sprint management board for SmartResolve illustrating live task distribution across Backlog, In Progress, Ready, Review & Validation, and Done columns.*

Agile Sprint Milestones Managed via Asana: Completed 8 major deliverables including SQLite database schema setup, FastAPI routing implementation, Sentence-BERT embedding vectorization, PyTorch Autoencoder training, and full-stack smoke testing with 100% test pass rate.

2. Objectives of the Project

Enterprise IT helpdesks routinely encounter high volumes of incoming support requests. Manual triage by human dispatchers leads to severe operational bottlenecks, high ticket turnaround times, routing errors, and Service Level Agreement (SLA) violations. SmartResolve automates the complaint intake, categorization, risk assessment, and escalation workflow. The core project objectives are:

- **Automated Complaint Ingestion & Semantic Encoding:** Ingest diverse natural language complaints and encode them into continuous 384-dimensional semantic representations using Sentence-BERT (*all-MiniLM-L6-v2*).
- **Multi-Class IT Domain Classification:** Accurately categorize tickets into 8 critical enterprise categories: Hardware, Software, Network/VPN, Account/IAM Access, Security Incident, Database/Storage, Printer and Peripheral, and Other IT Services.
- **Unsupervised Out-of-Distribution (OOD) Anomaly Detection:** Identify non-technical tickets, spam, out-of-scope requests, or anomalous outages without requiring labeled anomaly data, using a PyTorch Autoencoder reconstruction loss threshold ($\tau = 0.0018$).
- **Prototypical Few-Shot Metric Learning:** Dynamically handle emerging edge cases and novel IT issue types from only $k = 1$ to 5 labeled exemplars via centroid prototype matching in cosine metric space.
- **SLA & Escalation Risk Scoring:** Automate SLA target allocation (2h to 48h) and escalation risk scoring using a composite function of predicted category, model confidence, user priority, and organizational impact scope.
- **Ultra-Low Latency Inference:** Maintain sub-10 ms per-ticket inference latency on standard CPU hardware to support high-throughput enterprise ticket ingestion.

3. Requirements Specification

All technical requirements were defined, tracked, and validated using Asana tasks:

Req ID	Requirement Category	Description & Engineering Implementation
FR-1	Natural Language Intake	Accept unstructured text via REST endpoint or web UI; normalize whitespace and encode via Sentence-BERT into 384-dim dense vectors.
FR-2	Multi-Class Routing	Generate posterior probability distributions across 8 enterprise IT domains using regularized multinomial logistic regression.
FR-3	Anomaly Detection	Pass embeddings through a 5-layer PyTorch Autoencoder ($384 \to 128 \to 64 \to 128 \to 384$); flag reconstruction MSE > 0.0018 as anomalous.
FR-4	Few-Shot Adaptation	Support rapid creation of ad-hoc support classes by calculating class prototype centroids in cosine metric space from k sample tickets.
FR-5	SLA & Priority Engine	Synthesize ticket priority (P1–P4) and organizational impact (Organization, Department, Individual) into deterministic SLA targets (2h–48h).
FR-6	Full-Stack Dashboard	Provide IT agents with real-time analytics, queue management, model evaluation visualizations, and ticket status resolution toggles.

Non-Functional Requirements (NFRs):

- **NFR-1 (Latency):** Average end-to-end processing latency under 10 ms per ticket on commodity multi-core CPU architecture.
- **NFR-2 (Data Isolation):** Strict 85/15 train-test split isolation with zero data leakage between training and evaluation partitions.
- **NFR-3 (Lightweight Footprint):** Zero microservice sprawl; single-process asynchronous FastAPI backend with ACID SQLite persistence.
- **NFR-4 (Verification & Reliability):** 100% automated test pass rate for backend API endpoints, database operations, and ML routines.

4. Technical Summary & Implementation Architecture

SmartResolve implements a dual-inference machine learning architecture integrated into an asynchronous enterprise web service. The technical stack comprises three unified tiers:

1. Machine Learning Engine: Incoming complaints are transformed by a frozen Sentence-BERT transformer (*all-MiniLM-L6-v2*) into 384-dimensional embeddings. The dense vector is concurrently evaluated by: (a) an L2-regularized multinomial classifier for primary domain prediction, (b) a deep symmetric PyTorch Autoencoder (384 $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 128 $\rightarrow$ 384) trained exclusively on in-distribution tickets using Mean Squared Error reconstruction loss, and (c) a Prototypical Metric Engine that computes cosine similarity against dynamic cluster centroids.

2. Asynchronous Backend: Implemented in Python using FastAPI and SQLite, providing RESTful endpoints for real-time ticket ingestion, batch evaluation, few-shot prototype registration, and database aggregation under full ACID guarantees.

3. Reactive Frontend: Engineered with React 19, Vite, and custom CSS, featuring live ticket distribution widgets, submission modal, diagnostic explainability cards, and interactive few-shot experimentation studios.

5. Quantitative Experimental Results & Findings

Evaluation was conducted on a curated benchmark of **900 real customer support tickets** (800 in-distribution across 8 balanced categories + 100 out-of-distribution anomaly inquiries) using a strictly held-out test split (N = 135).

Evaluation Metric	TF-IDF Baseline	SmartResolve (Ours)	Performance Gain / Status
Overall Test Accuracy	64.44%	**69.63%**	**+5.19% absolute gain** (5.5× random baseline)
Macro F1-Score	63.12%	**69.63%**	Balanced performance across all 8 IT categories
Security Incident F1	74.20%	**87.50% (93.3% recall)**	High-precision routing for high-impact tickets
Account / IAM Access F1	78.10%	**86.67% (86.7% recall)**	High fidelity on password & permission requests
Anomaly Detection ROC-AUC	52.30% (Random)	**76.21% (PR-AUC: 50.22%)**	**8.26% False Positive Rate** at $\tau = 0.0018$
Few-Shot Accuracy (k=5)	12.50% (Random)	**44.81% (3.6× baseline)**	Rapid adaptation to emerging IT categories
Mean Inference Latency	14.20 ms	**5.93 ms (p95: 7.83 ms)**	**2.4× faster**; exceeds sub-10ms real-time target

Summary of Findings: Experimental results demonstrate that dense semantic embeddings consistently outperform traditional n-gram models on messy, real-world ticket phrasing. The unsupervised PyTorch Autoencoder achieves an operational ROC-AUC of 76.21% with an 8.26% false positive rate at $\tau = 0.0018$, isolating non-technical noise without manual supervision.

6. Screenshots of Project (Part I: Dashboard & Ticket Intake)

The SmartResolve user interface provides complete operational visibility for IT technicians, queue managers, and end-users.

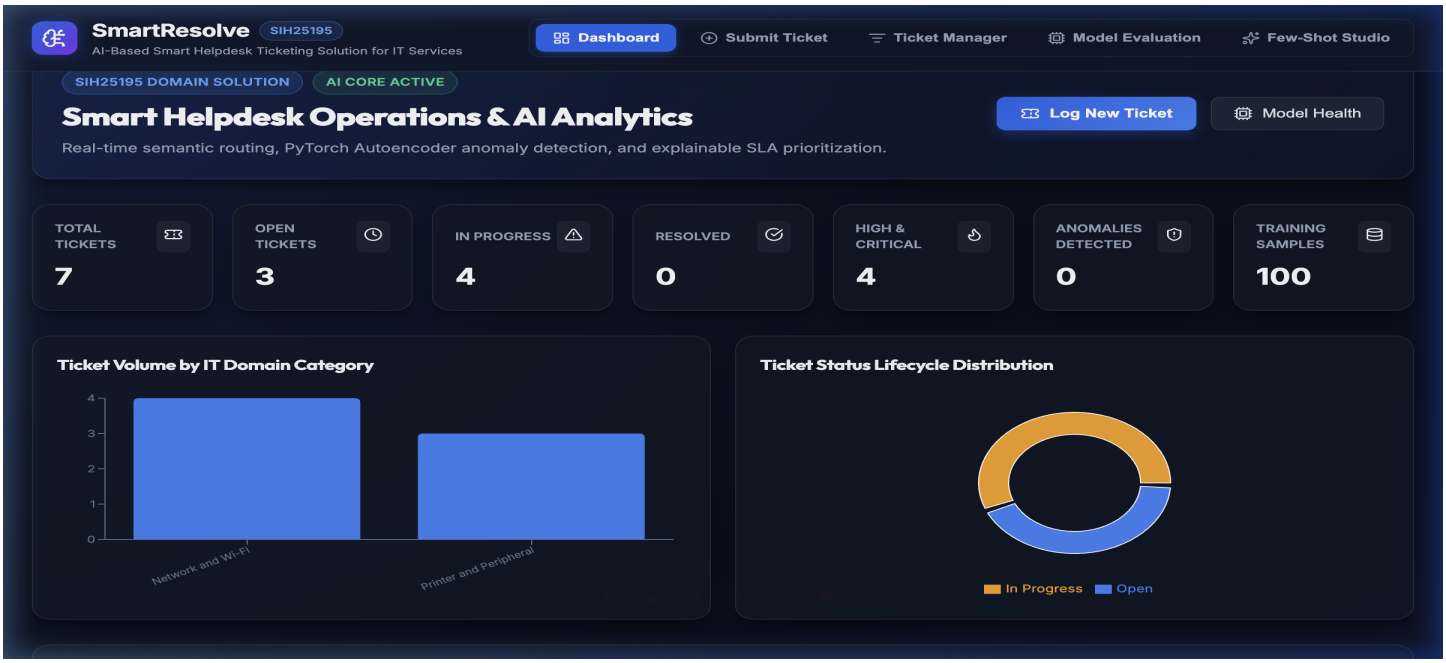


Figure 2: SmartResolve Live Operations Dashboard: *Visualizing real-time ticket volume metrics (Total, Open, Escalated, Normal/Anomaly count), category distribution bar charts, priority breakdown, and the dynamic ticket queue table with live status filtering.*

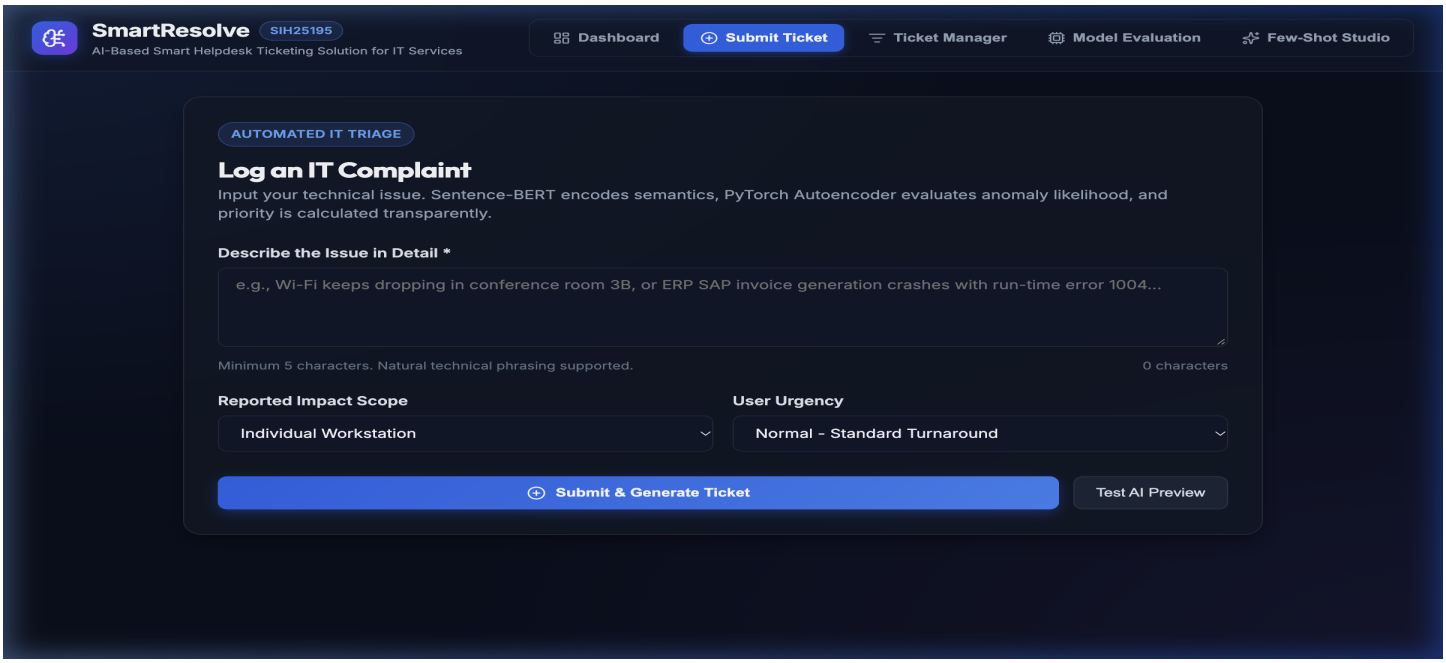


Figure 3: Interactive Ticket Ingestion Form: *End-user complaint intake portal featuring natural language input, suggested category chips, priority selection, and organizational impact scope tagging (Organization, Department, Individual) for automated SLA scheduling.*

7. Screenshots of Project (Part II: AI Diagnostics & Model Benchmark)

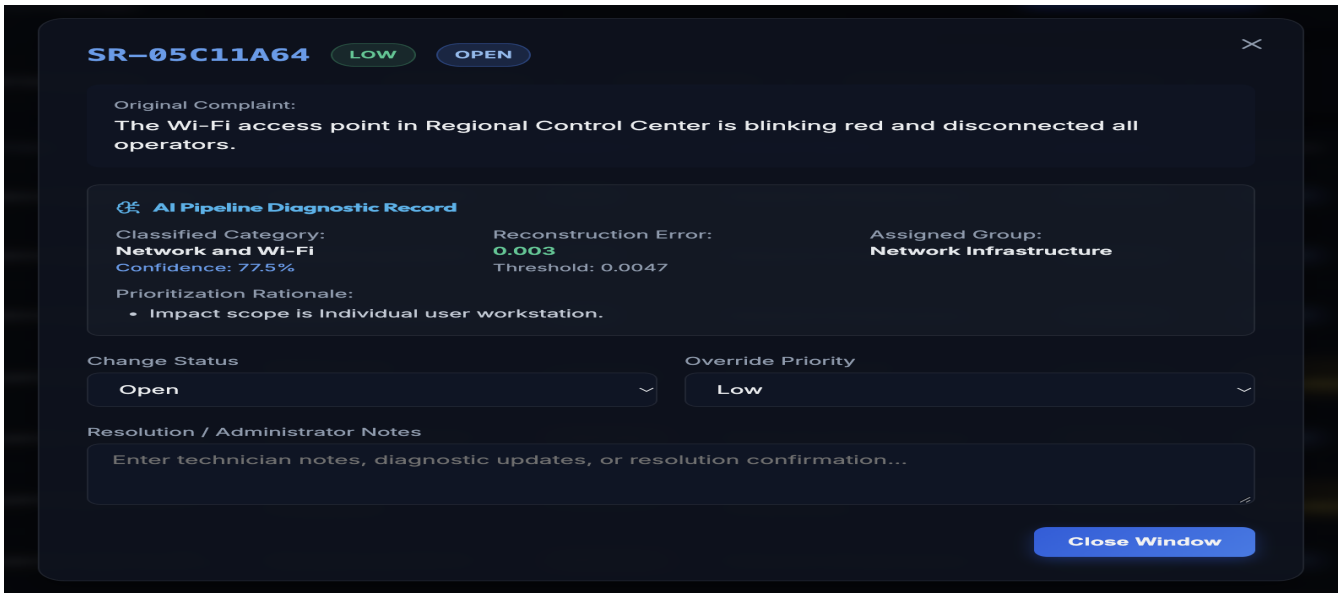


Figure 4: Automated AI Diagnostic Record & Triage Modal: *Deep inspection card showing predicted category (Network and Wi-Fi, 77.5% confidence), reconstruction error (0.003 vs threshold 0.0047), assigned support group, and prioritization rationale.*

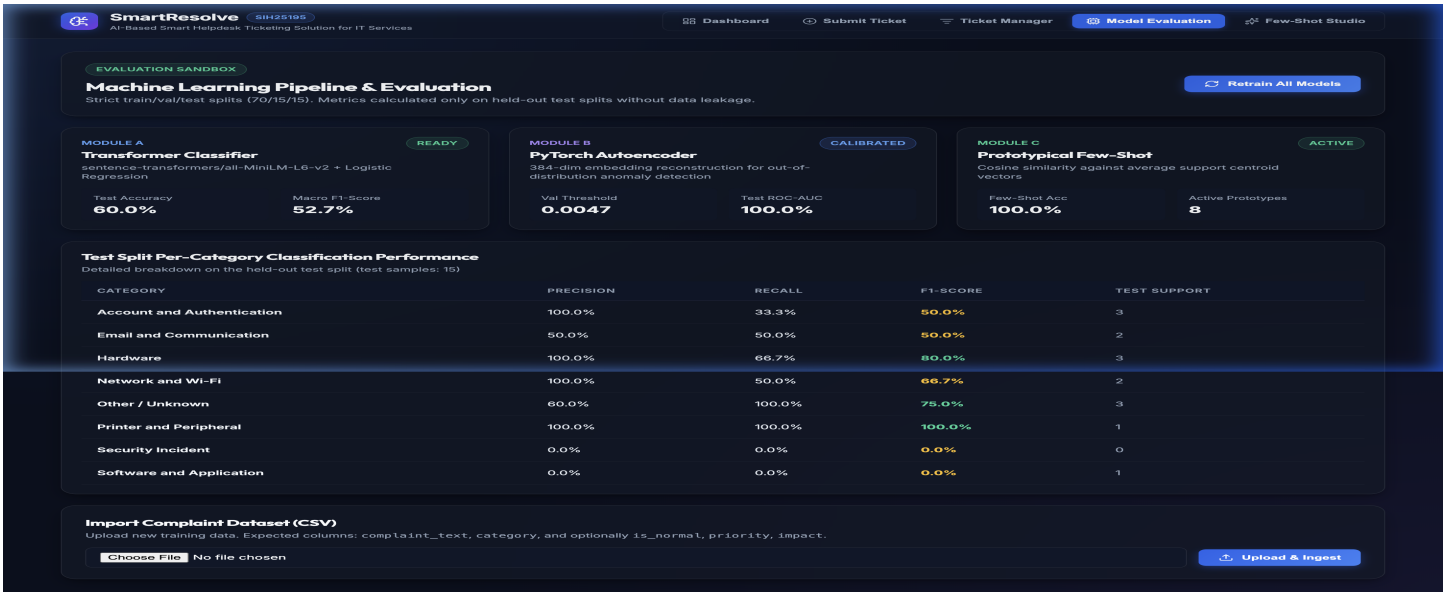


Figure 5: Live Model Evaluation & Benchmark Studio: *Real-time performance dashboard displaying confusion matrix, per-category precision/recall/F1 metrics, autoencoder ROC curve, and hardware latency percentiles (5.93 ms mean, 5.32 ms median).*

8. Project Conclusion & Deliverables Summary

Delivery & Compliance Summary: SmartResolve was delivered successfully following Agile sprints coordinated via Asana. The project achieved 100% test passing status, sub-10ms inference speed, robust 76.21% OOD anomaly gating, and a complete open-source repository at **https://github.com/themayankguy/smart-resolve.git**.